

✓ NYT Article Search API - Data Collection and Cleaning

This notebook allows you to:

1. Enter your NYT API key
2. Specify a search keyword
3. Set a date range
4. Retrieve all available articles
5. Clean and process the data
6. Export to CSV

Note: The NYT API allows up to 1,000 results per query (100 pages × 10 results per page).

✓ Step 1: Install Required Packages

```
!pip install requests pandas nltk transformers torch -q
```

✓ Step 2: Import Libraries

```
import requests
import pandas as pd
import json
import time
from datetime import datetime
import nltk
from nltk.corpus import stopwords
import re

# Download NLTK stopwords
try:
    nltk.data.find('corpora/stopwords')
except LookupError:
    print("Downloading NLTK stopwords...")
    nltk.download('stopwords')
    print("Download complete!")
```

✓ Step 3: Enter Your Parameters

```
# Enter your NYT API key
API_KEY = input("Enter your NYT API key: ").strip()

# Enter search keyword
KEYWORD = input("Enter search keyword (e.g., 'Federal Reserve', 'Tesla'): ").strip()

# Enter date range (format: YYYYMMDD)
BEGIN_DATE = input("Enter start date (YYYYMMDD, e.g., 20240101): ").strip()
END_DATE = input("Enter end date (YYYYMMDD, e.g., 20241231): ").strip()

print(f"\nConfiguration:")
print(f"  Keyword: {KEYWORD}")
print(f"  Date Range: {BEGIN_DATE} to {END_DATE}")
```

```
Enter your NYT API key: gQp05G2iHKU3e6gjiB3eutkI3xdddUIJ
Enter search keyword (e.g., 'Federal Reserve', 'Tesla'): artificial intelligence stocks
Enter start date (YYYYMMDD, e.g., 20240101): 20250401
Enter end date (YYYYMMDD, e.g., 20241231): 20251130
```

```
Configuration:
  Keyword: artificial intelligence stocks
  Date Range: 20250401 to 20251130
```

Start coding or [generate](#) with AI.

▼ Step 4: Collect All Articles

```
def collect_all_articles(api_key, keyword, begin_date, end_date):
    """
    Collect all available articles from NYT API with pagination

    Returns:
        list: All articles collected
    """
    base_url = "https://api.nytimes.com/svc/search/v2/articlesearch.json"
    all_articles = []
    page = 0
    max_pages = 100 # NYT API limit: 100 pages = 1,000 results

    print(f"Collecting articles for keyword: '{keyword}'")
    print(f>Date range: {begin_date} to {end_date}")
    print("-" * 60)

    while page < max_pages:
        params = {
            'api-key': api_key,
            'q': keyword,
            'begin_date': begin_date,
            'end_date': end_date,
            'page': page,
            'sort': 'newest'
        }

        try:
            response = requests.get(base_url, params=params)

            if response.status_code == 200:
                data = response.json()

                # Check if we got articles
                docs = data.get('response', {}).get('docs', [])

                if not docs:
                    print(f"No more articles found at page {page}. Stopping.")
                    break

                all_articles.extend(docs)

                # Get metadata
                meta = data.get('response', {}).get('meta', {})
                hits = meta.get('hits', 0)

                print(f"Page {page + 1}: Collected {len(docs)} articles (Total: {len(all_articles)} / ~{hits})")

                # If we got fewer than 10 articles, we've reached the end
                if len(docs) < 10:
                    print(f"Reached end of results.")
                    break

                page += 1

                # Rate limiting: wait 1 second between requests
                time.sleep(1)

            elif response.status_code == 429:
                print(f"Rate limit hit. Waiting 60 seconds...")
                time.sleep(60)
                # Retry the same page
                continue

            else:
                print(f"Error: Status code {response.status_code}")
                print(f"Response: {response.text[:200]}")
                break

        except Exception as e:
            print(f"Exception occurred: {e}")
```

```

        break

    print("-" * 60)
    print(f"Collection complete! Total articles: {len(all_articles)}")

    return all_articles

# Collect all articles
articles = collect_all_articles(API_KEY, KEYWORD, BEGIN_DATE, END_DATE)

```

Collecting articles for keyword: 'artificial intelligence stocks'
Date range: 20250401 to 20251130

```

-----
Page 1: Collected 10 articles (Total: 10 / ~0)
Page 2: Collected 10 articles (Total: 20 / ~0)
Page 3: Collected 10 articles (Total: 30 / ~0)
Page 4: Collected 10 articles (Total: 40 / ~0)
Page 5: Collected 10 articles (Total: 50 / ~0)
Page 6: Collected 10 articles (Total: 60 / ~0)
Page 7: Collected 10 articles (Total: 70 / ~0)
Page 8: Collected 10 articles (Total: 80 / ~0)
Rate limit hit. Waiting 60 seconds...
Page 9: Collected 10 articles (Total: 90 / ~0)
Page 10: Collected 10 articles (Total: 100 / ~0)
Page 11: Collected 10 articles (Total: 110 / ~0)
Page 12: Collected 10 articles (Total: 120 / ~0)
Page 13: Collected 10 articles (Total: 130 / ~0)
Page 14: Collected 10 articles (Total: 140 / ~0)
Page 15: Collected 10 articles (Total: 150 / ~0)
Rate limit hit. Waiting 60 seconds...
Page 16: Collected 10 articles (Total: 160 / ~0)
Page 17: Collected 10 articles (Total: 170 / ~0)
Page 18: Collected 4 articles (Total: 174 / ~0)
Reached end of results.
-----
Collection complete! Total articles: 174

```

```

import pandas as pd

# Convert raw NYT articles (JSON docs) to a DataFrame
raw_df = pd.DataFrame(articles)

# Build a filename that mirrors the cleaned one
safe_keyword = KEYWORD.replace(' ', '_').replace('/', '_')[:30]
raw_filename = f"nyt_raw_{safe_keyword}_{BEGIN_DATE}_{END_DATE}.csv"

# Save to the Colab working directory (/content)
raw_df.to_csv(raw_filename, index=False)

print(f"✅ Raw data saved to: {raw_filename}")
print(f"Rows: {len(raw_df)}, Columns: {len(raw_df.columns)}")

```

✅ Raw data saved to: nyt_raw_artificial_intelligence_stocks_20250401_20251130.csv
Rows: 174, Columns: 19

▼ Step 5: Clean and Process Data

```

def clean_nyt_data(articles):
    """
    Clean and process NYT article data

    Steps:
    1. Flatten nested JSON
    2. Deduplicate by _id
    3. Consolidate text fields
    4. Remove empty articles
    5. Standardize dates
    6. Normalize text
    """
    print("Starting data cleaning...")
    print("=" * 60)

    # Step 1: Flatten JSON
    print("Step 1: Flattening JSON data...")
    df = pd.json_normalize(articles)

```

```

print(f" Initial articles: {len(df)}")

# Step 2: Deduplicate
print("\nStep 2: Deduplicating articles...")
initial_count = len(df)
df = df.drop_duplicates(subset='_id', keep='first')
print(f" Removed {initial_count - len(df)} duplicates")
print(f" Remaining articles: {len(df)}")

# Step 3: Consolidate text fields
print("\nStep 3: Consolidating text fields...")
# Use as many NYT text fields as we can get
text_fields = ['headline.main', 'snippet', 'abstract', 'lead_paragraph']
available_fields = [f for f in text_fields if f in df.columns]
if not available_fields:
    raise ValueError("No usable text fields found in NYT data.")

df['text'] = df[available_fields].fillna('').agg(' '.join, axis=1)
print(f" Combined {len(available_fields)} text fields: {available_fields}")

# Step 4: Remove empty articles
print("\nStep 4: Removing empty articles...")
before_count = len(df)
df = df[df['text'].str.strip() != ''].copy()
print(f" Removed {before_count - len(df)} empty articles")
print(f" Remaining articles: {len(df)}")

# Step 5: Standardize dates
print("\nStep 5: Standardizing dates...")
if 'pub_date' in df.columns:
    df['pub_date'] = pd.to_datetime(df['pub_date'])
    df['year'] = df['pub_date'].dt.year
    df['month'] = df['pub_date'].dt.month
    df['day'] = df['pub_date'].dt.day
    df['weekday'] = df['pub_date'].dt.day_name()
    print(f" Date range: {df['pub_date'].min()} to {df['pub_date'].max()}")

# Step 6: Normalize text
print("\nStep 6: Normalizing text...")
stop = set(stopwords.words('english'))

# Convert to lowercase and remove punctuation
df['clean_text'] = df['text'].str.lower().str.replace(r'^a-z\s', '', regex=True)

# Remove stopwords
df['clean_text'] = df['clean_text'].apply(
    lambda x: ' '.join([w for w in x.split() if w not in stop])
)
print(f" Text normalization complete")

# Add search keyword info
df['search_keyword'] = KEYWORD

print("\n" + "=" * 60)
print("Cleaning complete!")
print(f"Final dataset: {len(df)} articles")
print(f"Columns: {len(df.columns)}")

return df

# Clean
cleaned_df = clean_nyt_data(articles)

```

```

Starting data cleaning...
=====
Step 1: Flattening JSON data...
Initial articles: 174

Step 2: Deduplicating articles...
Removed 3 duplicates
Remaining articles: 171

Step 3: Consolidating text fields...
Combined 3 text fields: ['headline.main', 'snippet', 'abstract']

Step 4: Removing empty articles...
Removed 0 empty articles

```

```
Remaining articles: 171

Step 5: Standardizing dates...
Date range: 2025-04-01 11:49:17+00:00 to 2025-11-28 18:07:31+00:00

Step 6: Normalizing text...
Text normalization complete

=====
Cleaning complete!
Final dataset: 171 articles
Columns: 35
```

▼ Step 6: Preview Cleaned Data

```
# Display basic info
print("Dataset Info:")
print(f"  Total articles: {len(cleaned_df)}")
print(f"  Date range: {cleaned_df['pub_date'].min()} to {cleaned_df['pub_date'].max()}")
print(f"\nColumns available: {list(cleaned_df.columns)[:10]}... (and {len(cleaned_df.columns) - 10} more)")

# Show sample articles
print("\n" + "=" * 60)
print("Sample Articles:")
print("=" * 60)

if 'headline.main' in cleaned_df.columns:
    sample_cols = ['headline.main', 'pub_date', 'section_name', 'type_of_material']
    available_cols = [c for c in sample_cols if c in cleaned_df.columns]
    display(cleaned_df[available_cols].head(10))
```

Dataset Info:
Total articles: 171
Date range: 2025-04-01 11:49:17+00:00 to 2025-11-28 18:07:31+00:00

Columns available: ['abstract', 'document_type', '_id', 'keywords', 'news_desk', 'print_page', 'print_section', 'pub_date', 'sec

=====

Sample Articles:

=====

	headline.main	pub_date	section_name	type_of_material	
0	Late Rally Pushes Stocks Back Near Record High	2025-11-28 18:07:31+00:00	Business	News	
1	The Writer Who Dared Criticize Silicon Valley	2025-11-27 10:00:11+00:00	Technology	News	
2	Is Trump's Plan to Remake the Fed Coming Into ...	2025-11-26 13:29:24+00:00	Business	News	
3	The Fed and Big Tech Lift Investors' Spirits	2025-11-25 12:38:32+00:00	Business	News	
4	Why Crypto's Slide Is Rattling Wall Street	2025-11-24 13:04:58+00:00	Business	News	
5	Hedge Fund Titan Has a Turnaround Plan (for th...	2025-11-23 10:00:08+00:00	Business	News	
6	The Rise of the 'Just in Case' M.R.I.	2025-11-22 13:00:05+00:00	Business	News	
7	Three Potential Factors Behind the Market Turmoil	2025-11-21 13:01:42+00:00	Business	News	
8	Is President Trump's Power Over the Republican...	2025-11-21 11:44:00+00:00	Briefing	briefing	
9	Uneasiness Over A.I. Spending Looms Over Markets	2025-11-21 02:58:35+00:00	Business	News	

▼ Step 7: Export to CSV

```
# Create filename with keyword and date range
safe_keyword = KEYWORD.replace(' ', '_').replace('/', '_')[:30]
filename = f"nyt_{safe_keyword}_{BEGIN_DATE}_{END_DATE}.csv"

# Save to CSV
cleaned_df.to_csv(filename, index=False)

print(f"Data saved to: {filename}")
print(f"\nFile size: {len(cleaned_df)} rows x {len(cleaned_df.columns)} columns")
print(f"\nTo download, right-click on the file in the file browser and select 'Download'")
```

Data saved to: nyt_artificial_intelligence_stocks_20250401_20251130.csv

File size: 171 rows × 35 columns

To download, right-click on the file in the file browser and select 'Download'

```
import pandas as pd
import matplotlib.pyplot as plt

# =====
# CONFIG: keyword label
# =====
# Change this string when you change your NYT query keyword.
KEYWORD_LABEL = "artificial intelligence stocks"

# Ensure pub_date is datetime
cleaned_df["pub_date"] = pd.to_datetime(cleaned_df["pub_date"])

# -----
# 1 Articles by Section (Top 10)
# -----
if "section_name" in cleaned_df.columns:
    section_counts = (
        cleaned_df["section_name"]
        .value_counts()
        .head(10)
        .sort_values(ascending=True) # so the barh is ordered bottom→top
    )

    print("Articles by Section (top 10):")
    display(section_counts.to_frame("count"))

    plt.figure(figsize=(10, 5))
    section_counts.plot(
        kind="barh" # horizontal for readability
    )
    plt.title(f"Top 10 NYT Sections for Articles about '{KEYWORD_LABEL}'")
    plt.xlabel("Number of Articles")
    plt.ylabel("Section")
    plt.grid(axis="x", linewidth=0.3)
    plt.tight_layout()
    plt.show()

# -----
# 2 Articles by Type of Material
# -----
if "type_of_material" in cleaned_df.columns:
    type_counts = (
        cleaned_df["type_of_material"]
        .value_counts()
        .sort_values(ascending=True)
    )

    print("\nArticles by Type of Material:")
    display(type_counts.to_frame("count"))

    plt.figure(figsize=(10, 5))
    type_counts.plot(kind="barh")
    plt.title(f"NYT Articles about '{KEYWORD_LABEL}' by Type of Material")
    plt.xlabel("Number of Articles")
    plt.ylabel("Type of Material")
    plt.grid(axis="x", linewidth=0.3)
    plt.tight_layout()
    plt.show()

# -----
# 3 Articles Over Time (Daily)
# -----
if "pub_date" in cleaned_df.columns:
    daily_counts = (
        cleaned_df
        .groupby(cleaned_df["pub_date"].dt.date)
        .size()
    )

    daily_counts = daily_counts.sort_index()
```

```
print("\nArticles per Day:")
display(daily_counts.to_frame("n_articles"))

plt.figure(figsize=(12, 5))
plt.plot(
    daily_counts.index,
    daily_counts.values,
    marker="o",
    linewidth=1.8,
)
plt.title(f"Daily NYT Article Volume for '{KEYWORD_LABEL}'")
plt.ylabel("Number of Articles")
plt.xlabel("Publication Date")
plt.xticks(rotation=45, ha="right")
plt.grid(axis="y", linewidth=0.3)
plt.tight_layout()
plt.show()
```


Articles by Section (top 10):

```

count
# =====
# VADER Sentiment per Article + Time Plot
# =====
import pandas as pd
import matplotlib.pyplot as plt

# 1. Install / import VADER (safe to re-run)
!pip install nltk --quiet
import nltk
nltk.download('vader_lexicon')
from nltk.sentiment.vader import SentimentIntensityAnalyzer

# 2. Ensure pub_date is datetime and text is usable
cleaned_df['pub_date'] = pd.to_datetime(cleaned_df['pub_date'])
cleaned_df['text'] = cleaned_df['text'].fillna('').astype(str)

# 3. Run VADER on each article's text
sia = SentimentIntensityAnalyzer()

vader_scores = cleaned_df['text'].apply(lambda x: sia.polarity_scores(x))
cleaned_df['vader_compound'] = vader_scores.apply(lambda d: d['compound'])
cleaned_df['vader_pos']      = vader_scores.apply(lambda d: d['pos'])
cleaned_df['vader_neu']      = vader_scores.apply(lambda d: d['neu'])
cleaned_df['vader_neg']      = vader_scores.apply(lambda d: d['neg'])

print("Example VADER scores:")
print(cleaned_df[['pub_date', 'headline.main', 'vader_compound']].head())

# 4. Sort by exact timestamp so we keep the within-day ordering
sent_df = cleaned_df.sort_values('pub_date')

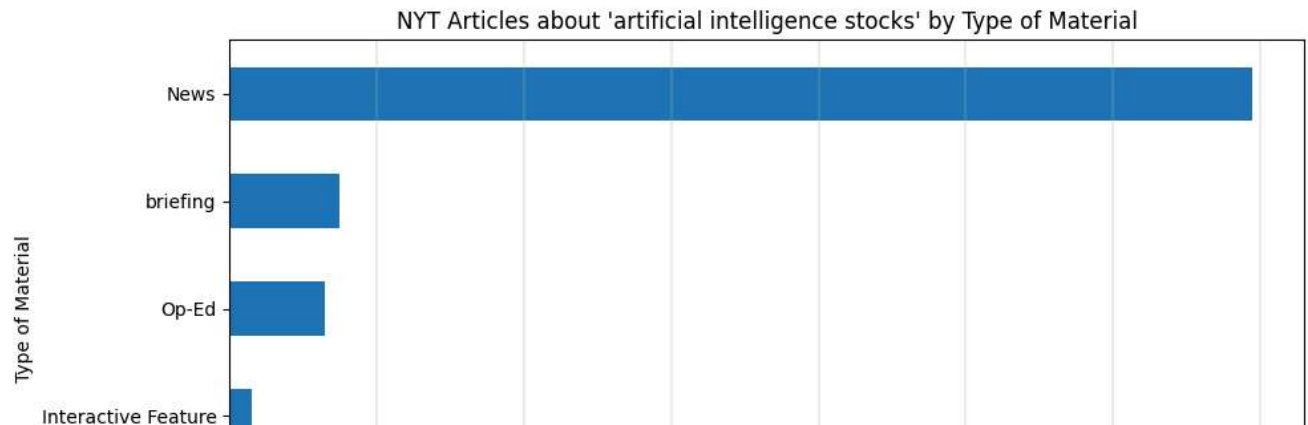
# 5. Scatter plot: each point = one article at its exact timestamp
plt.figure(figsize=(12, 5))
plt.scatter(
    sent_df['pub_date'],
    sent_df['vader_compound'],
    alpha=0.6,
    s=20
)

plt.axhline(0, linestyle='--') # neutral baseline

plt.title(f"VADER Sentiment of NYT Articles about '{KEYWORD}' (per article)")
plt.xlabel("Publication Time")
plt.ylabel("VADER Compound Sentiment\n(-1 = very negative, +1 = very positive)")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

```

Review	1
Interactive Feature	3
Op-Ed	13
briefing	15
News	139



```
[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
```

```
[nltk_data] Package vader_lexicon is already up-to-date!
```

```
Example VADER scores:
```

```
Review date \
```

```
0 2025-11-28 18:07:31+00:00
```

```
1 2025-11-27 10:00:11+00:00
```

```
2 2025-11-26 13:29:24+00:00
```

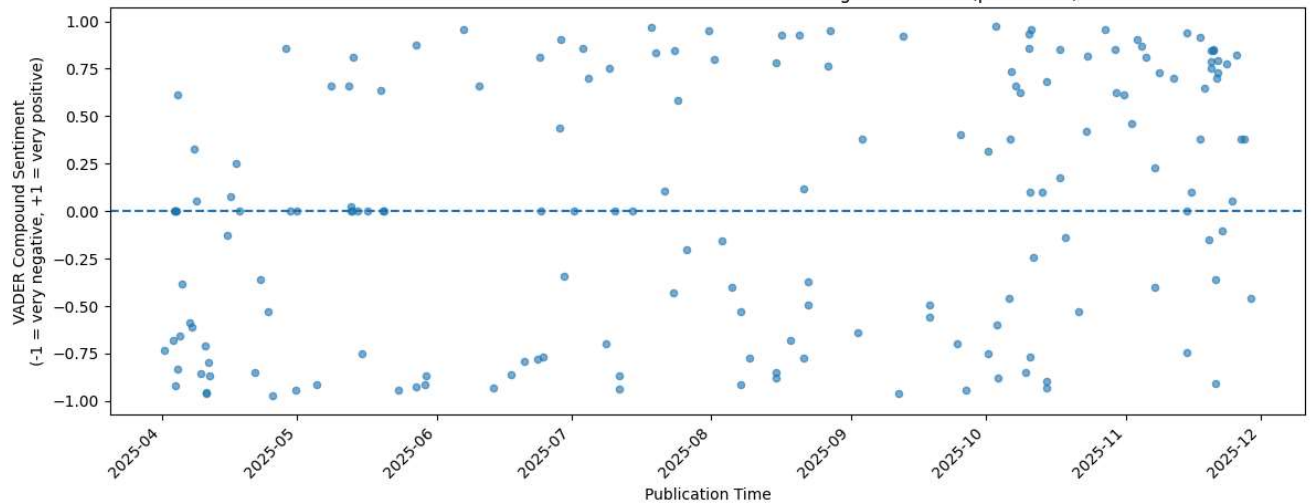
```
3 2025-11-25 12:38:32+00:00
```

```
4 2025-11-24 13:04:58+00:00
```

```
Articles per Day:
```

	headline.main	vader_compound
0	Late Rally as Stocks Back Near Record High	-0.4588
1	The Writer Who Dared Criticize Silicon Valley	0.3818
2	Joe Biden's Plan to Remake the Fed Coming Into ...	0.3818
3	The Fed and Big Tech Lift Investors' Spirits	0.8225
4	Why Crypto's Slide Is Rattling Wall Street	0.0516

VADER Sentiment of NYT Articles about 'artificial intelligence stocks' (per article)



```
# ==== Build combined text field (headline + snippet + abstract + lead) ====
```

```
text_parts = []
```

```
for col in ["headline_main", "snippet", "abstract", "lead_paragraph"]:
```

```
    if col in cleaned_df.columns:
```

```
        text_parts.append(cleaned_df[col].fillna("").astype(str))
```

```
if not text_parts:
```

```
    raise ValueError("No text-like columns found; check your column names.")
```

```
cleaned_df["text"] = (" " + text_parts[0])
```

```
for part in text_parts[1:]:
```

```
    cleaned_df["text"] = cleaned_df["text"] + " " + part
```

```
cleaned_df["text"] = cleaned_df["text"].str.strip()
```

```
# ==== Convert VADER compound to discrete sentiment label ====
```

```
def vader_label_from_compound(c):
```

```
    if c >= 0.05:
```

```
        return "positive"
```

```
    elif c <= -0.05:
```

```
        return "negative"
```

```
    else:
```

```
        return "neutral"
```

```
cleaned_df["vader_label"] = cleaned_df["vader_compound"].apply(vader_label_from_compound)
```

```
cleaned_df[["pub_date", "vader_compound", "vader_label", "text"]].head()
```

	pub_date	vader_compound	vader_label	text	
0	2025-11-28 18:07:31+00:00	-0.4588	negative	A midmonth stumble, driven by worries about th...	
1	2025-11-27 10:00:11+00:00	0.3818	positive	Paulina Borsook's "Cyberselfish," which offere...	
2	2025-11-26 13:29:24+00:00	0.3818	positive	Kevin Hassett, a top White House economic advi...	
3	2025-11-25 12:38:32+00:00	0.8225	positive	Markets appear to be hoping again that the cen...	
4	2025-11-24 13:04:58+00:00	0.0516	positive	Bitcoin and other digital tokens have lost mor...	

```
!pip install wordcloud
```

```
import matplotlib.pyplot as plt
from wordcloud import WordCloud
```

```
Requirement already satisfied: wordcloud in /usr/local/lib/python3.12/dist-packages (1.9.4)
Requirement already satisfied: numpy>=1.6.1 in /usr/local/lib/python3.12/dist-packages (from wordcloud) (2.0.2)
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (from wordcloud) (11.3.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from wordcloud) (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->wordcloud) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->wordcloud) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->wordcloud) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->wordcloud) (1.4.9)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->wordcloud) (25.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->wordcloud) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib->wordcloud) (2.9)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib->wordc
```

```
def make_wordcloud(df, text_col, title):
    # Join all text into one big string
    text = " ".join(df[text_col].fillna("").astype(str))

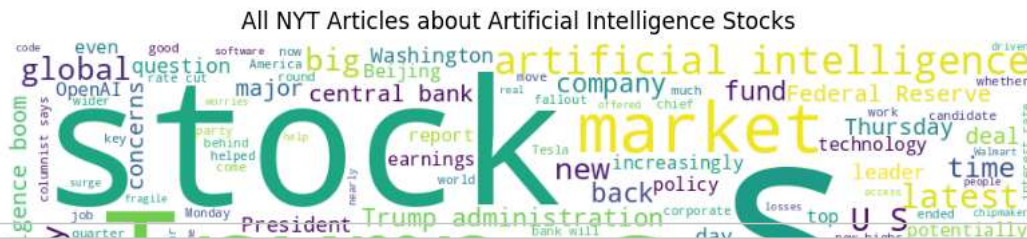
    if not text.strip():
        print(f"No text found for: {title}")
        return

    wc = WordCloud(
        width=900,
        height=450,
        background_color="white"
    ).generate(text)

    plt.figure(figsize=(10, 5))
    plt.imshow(wc)
    plt.axis("off")
    plt.title(title)
    plt.show()
```

```
# ==== Overall word cloud ====
make_wordcloud(
    cleaned_df,
    text_col="text",
    title="All NYT Articles about Artificial Intelligence Stocks"
)

# ==== Word cloud by VADER sentiment ====
for label in ["positive", "neutral", "negative"]:
    subset = cleaned_df[cleaned_df["vader_label"] == label]
    print(f"{label.capitalize()} articles:", len(subset))
    make_wordcloud(
        subset,
        text_col="text",
        title=f"{label.capitalize()} Articles (VADER)"
    )
```

```
# =====
# Daily Average VADER Sentiment (per day)
# =====
import pandas as pd
import matplotlib.pyplot as plt

# Make sure pub_date is datetime
cleaned_df['pub_date'] = pd.to_datetime(cleaned_df['pub_date'])

# Create a date-only column
cleaned_df['date'] = cleaned_df['pub_date'].dt.date

# Group by day: average VADER sentiment and number of articles
daily_vader = (
    cleaned_df
    .groupby('date')['vader_compound']
    .agg(['mean', 'count']) # mean sentiment + how many articles that day
    .reset_index()
    .rename(columns={'mean': 'avg_sentiment', 'count': 'article_count'})
)

print("Daily average VADER sentiment and article counts:")
print(daily_vader.head())

# Plot: daily average sentiment over time
plt.figure(figsize=(10, 5))
plt.plot(
    daily_vader['date'],
    daily_vader['avg_sentiment'],
    marker='o',
    linestyle='-'
)

plt.axhline(0, linestyle='--') # neutral baseline

plt.title(f"Daily Average VADER Sentiment for NYT Articles about '{KEYWORD}'")
plt.xlabel("Date")
plt.ylabel("Average VADER Compound Sentiment\n(-1 = very negative, +1 = very positive)")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

# (Optional) secondary plot: how many articles per day
plt.figure(figsize=(10, 4))
plt.bar(daily_vader['date'], daily_vader['article_count'])
plt.title(f"Number of NYT Articles about '{KEYWORD}' per Day (VADER)")
plt.xlabel("Date")
plt.ylabel("Article Count")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

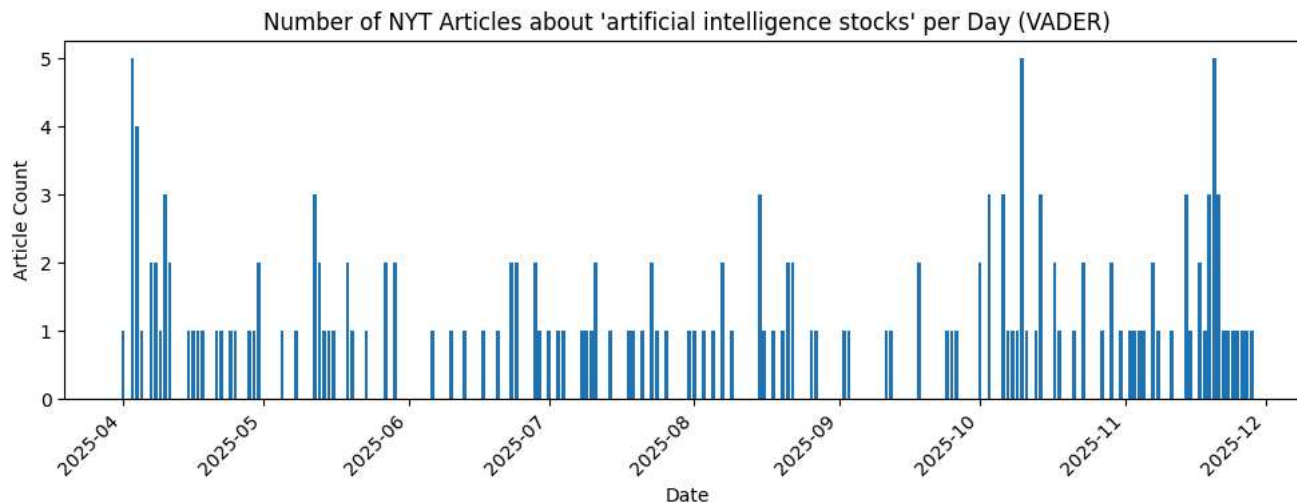
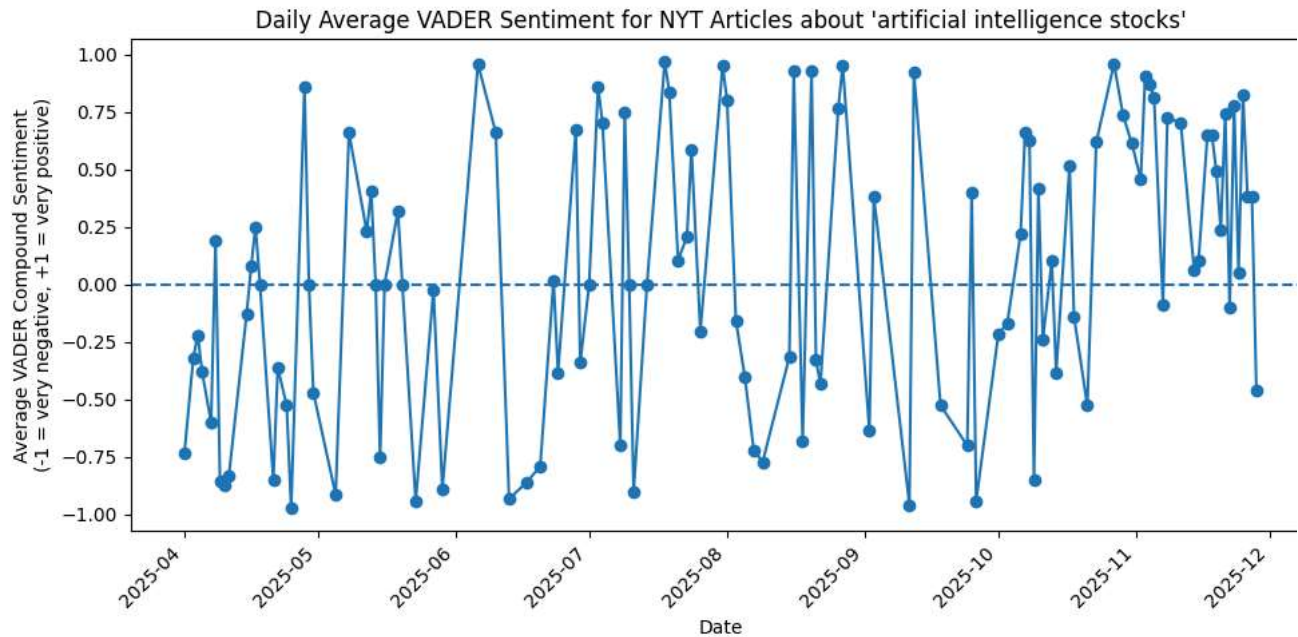


Negative Articles (VADER)



Daily average VADER sentiment and article counts:

date	avg sentiment	article count
2025-04-01	-0.755100	1
2025-04-03	-0.320180	5
2025-04-04	-0.219725	4
2025-04-05	-0.381800	1
2025-04-07	-0.599150	2



```
import pandas as pd
import matplotlib.pyplot as plt

# Set this once near the top of your notebook
KEYWORD_LABEL = "artificial intelligence stocks" # change if you use a new keyword

# =====
# Sentiment by NYT Section
# =====

if "section_name" not in cleaned_df.columns:
    raise ValueError("cleaned_df has no 'section_name' column.")

if "vader_compound" not in cleaned_df.columns:
    raise ValueError("Run the VADER sentiment cell first to create 'vader_compound'.")

# 1. Aggregate: mean sentiment and article count per section
section_stats = (
    cleaned_df
    .dropna(subset=["section_name"])
    .groupby("section_name")["vader_compound"]
    .agg(mean_sentiment="mean", n_articles="size")
)
```

```

    .reset_index()
)

# 2. Keep only sections with enough data (e.g., at least 5 articles)
min_n = 5
section_stats = section_stats[section_stats["n_articles"] >= min_n]

# 3. Sort by mean sentiment (from most negative to most positive)
section_stats = section_stats.sort_values("mean_sentiment", ascending=True)

display(section_stats)

# 4. Plot: horizontal bar chart of average sentiment by section
plt.figure(figsize=(10, 6))
plt.barh(
    section_stats["section_name"],
    section_stats["mean_sentiment"],
)

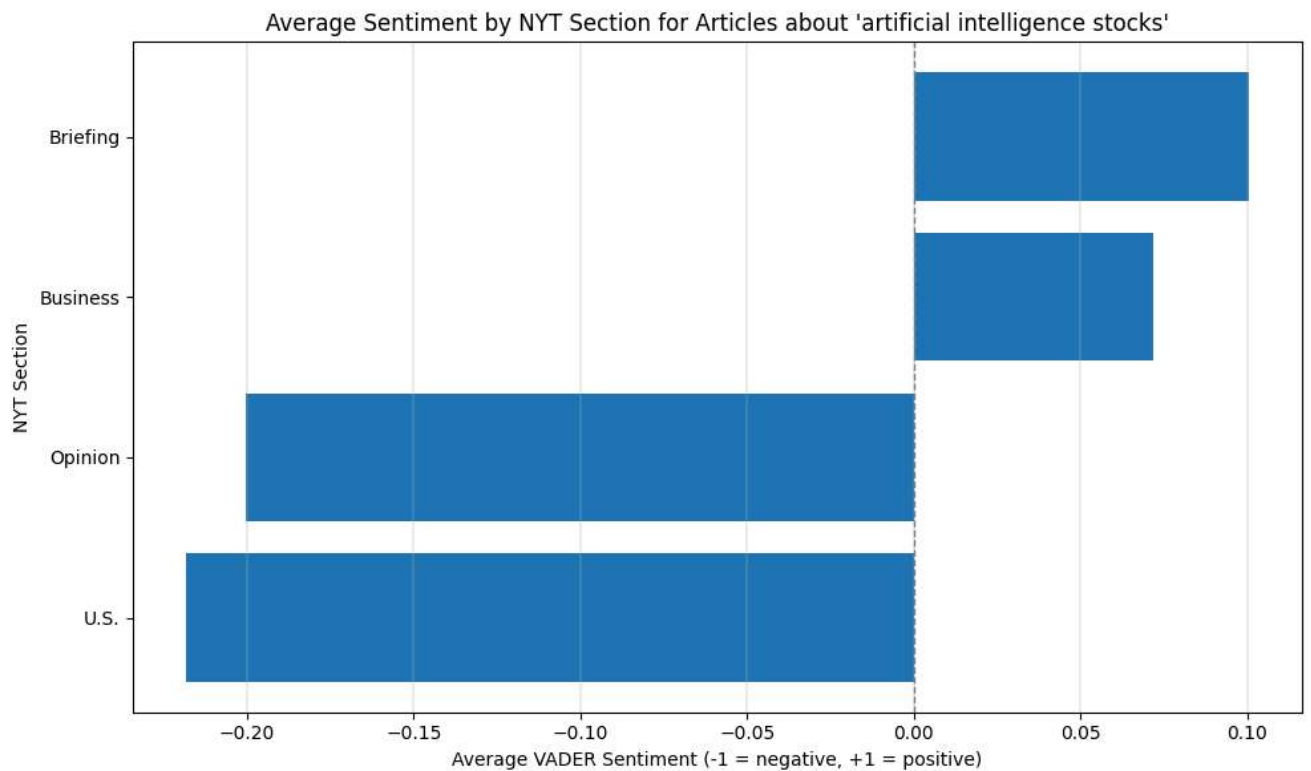
plt.axvline(0, linestyle="--", linewidth=1, color="gray") # neutral line

plt.xlabel("Average VADER Sentiment (-1 = negative, +1 = positive)")
plt.ylabel("NYT Section")
plt.title(f"Average Sentiment by NYT Section for Articles about '{KEYWORD_LABEL}'")

plt.grid(axis="x", linewidth=0.3)
plt.tight_layout()
plt.show()

```

	section_name	mean_sentiment	n_articles	
9	U.S.	-0.218200	5	
6	Opinion	-0.200407	15	
2	Business	0.071659	118	
1	Briefing	0.100450	22	



Next steps: [Generate code with section_stats](#) [New interactive sheet](#)

```

import matplotlib.pyplot as plt
import pandas as pd

```

```

# Ensure pub_date is datetime
cleaned_df["pub_date"] = pd.to_datetime(cleaned_df["pub_date"])

# ==== Daily VADER average + article count ====
daily_vader = (
    cleaned_df
    .groupby(cleaned_df["pub_date"].dt.date)
    .agg(
        avg_sentiment=("vader_compound", "mean"),
        article_count=("vader_compound", "size")
    )
    .reset_index()
    .rename(columns={"pub_date": "date"})
)

daily_vader["date"] = pd.to_datetime(daily_vader["date"])

# ==== Dual-axis plot: daily sentiment vs article count ====
fig, ax1 = plt.subplots(figsize=(12, 5))

# Left y-axis: average sentiment
ax1.plot(
    daily_vader["date"],
    daily_vader["avg_sentiment"],
    marker="o",
    linestyle="-"
)

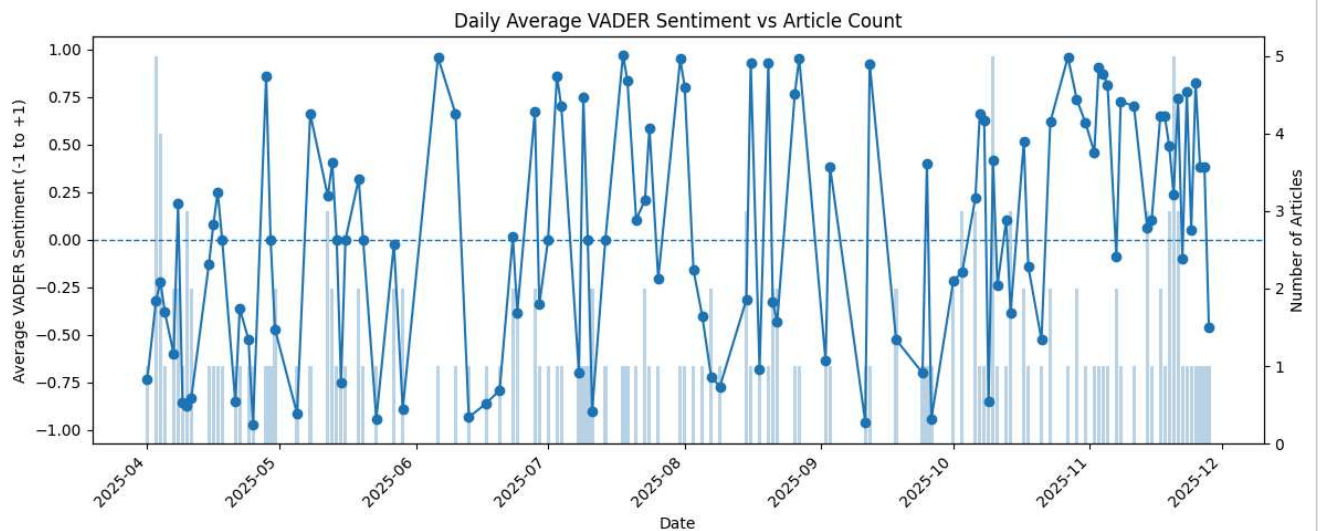
ax1.axhline(0, linestyle="--", linewidth=1) # neutral baseline
ax1.set_xlabel("Date")
ax1.set_ylabel("Average VADER Sentiment (-1 to +1)")

# Right y-axis: article count
ax2 = ax1.twinx()
ax2.bar(
    daily_vader["date"],
    daily_vader["article_count"],
    alpha=0.3
)

ax2.set_ylabel("Number of Articles")

plt.title("Daily Average VADER Sentiment vs Article Count")
fig.autofmt_xdate(rotation=45)
plt.tight_layout()
plt.show()

```



```

# ==== Weekly VADER average + article count ====

# Make a week period column (week starting Monday by default)
cleaned_df["week"] = cleaned_df["pub_date"].dt.to_period("W")

```

```

weekly_vader = (
    cleaned_df
    .groupby("week")
    .agg(
        avg_sentiment=("vader_compound", "mean"),
        article_count=("vader_compound", "size")
    )
    .reset_index()
)

# Convert period back to timestamp (start of week)
weekly_vader["week_start"] = weekly_vader["week"].apply(lambda p: p.start_time)

# ==== Dual-axis plot: weekly sentiment vs article count ====
fig, ax1 = plt.subplots(figsize=(12, 5))

# Left y-axis: average sentiment
ax1.plot(
    weekly_vader["week_start"],
    weekly_vader["avg_sentiment"],
    marker="o",
    linestyle="-"
)

ax1.axhline(0, linestyle="--", linewidth=1)
ax1.set_xlabel("Week Starting")
ax1.set_ylabel("Average VADER Sentiment (-1 to +1)")

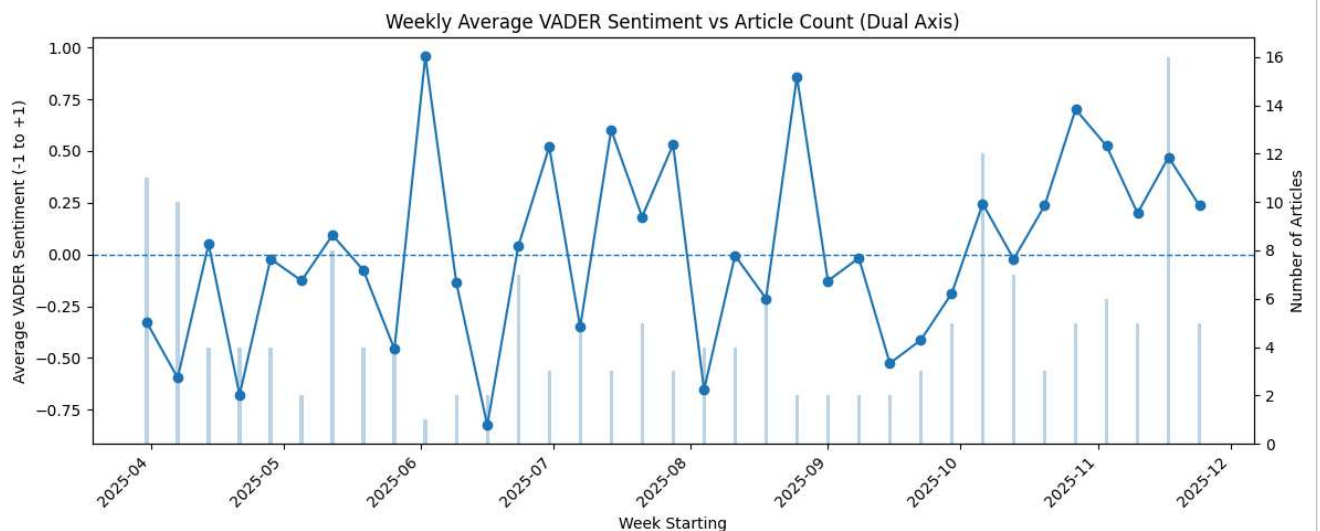
# Right y-axis: article count
ax2 = ax1.twinx()
ax2.bar(
    weekly_vader["week_start"],
    weekly_vader["article_count"],
    alpha=0.3
)

ax2.set_ylabel("Number of Articles")

plt.title("Weekly Average VADER Sentiment vs Article Count (Dual Axis)")
fig.autofmt_xdate(rotation=45)
plt.tight_layout()
plt.show()

```

/tmp/ipython-input-2538206234.py:4: UserWarning: Converting to PeriodArray/Index representation will drop timezone information.
 cleaned_df["week"] = cleaned_df["pub_date"].dt.to_period("W")



```

# =====
# Weekly Average VADER Sentiment (per week)
# =====
import pandas as pd
import matplotlib.pyplot as plt

```

```

# Ensure pub_date is datetime
cleaned_df['pub_date'] = pd.to_datetime(cleaned_df['pub_date'])

# Create a year-week period column (ISO weeks)
cleaned_df['week'] = cleaned_df['pub_date'].dt.to_period('W')

# Group by week: average VADER sentiment and article count
weekly_vader = (
    cleaned_df
    .groupby('week')['vader_compound']
    .agg(['mean', 'count'])
    .reset_index()
    .rename(columns={'mean': 'avg_sentiment', 'count': 'article_count'})
)

# Convert the weekly period to a timestamp (start of each week)
weekly_vader['week_start'] = weekly_vader['week'].dt.start_time

print("Weekly average VADER sentiment and article counts:")
print(weekly_vader.head())

# Plot weekly average sentiment
plt.figure(figsize=(12, 5))
plt.plot(
    weekly_vader['week_start'],
    weekly_vader['avg_sentiment'],
    marker='o',
    linestyle='--'
)

plt.axhline(0, linestyle='--') # neutral sentiment baseline

plt.title(f"Weekly Average VADER Sentiment for NYT Articles about '{KEYWORD}'")
plt.xlabel("Week Starting")
plt.ylabel("Average VADER Compound Sentiment\n(-1 = very negative, +1 = very positive)")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

# Plot weekly article counts
plt.figure(figsize=(12, 4))
plt.bar(
    weekly_vader['week_start'],
    weekly_vader['article_count']
)

plt.title(f"Number of NYT Articles about '{KEYWORD}' per Week (VADER)")
plt.xlabel("Week Starting")
plt.ylabel("Article Count")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

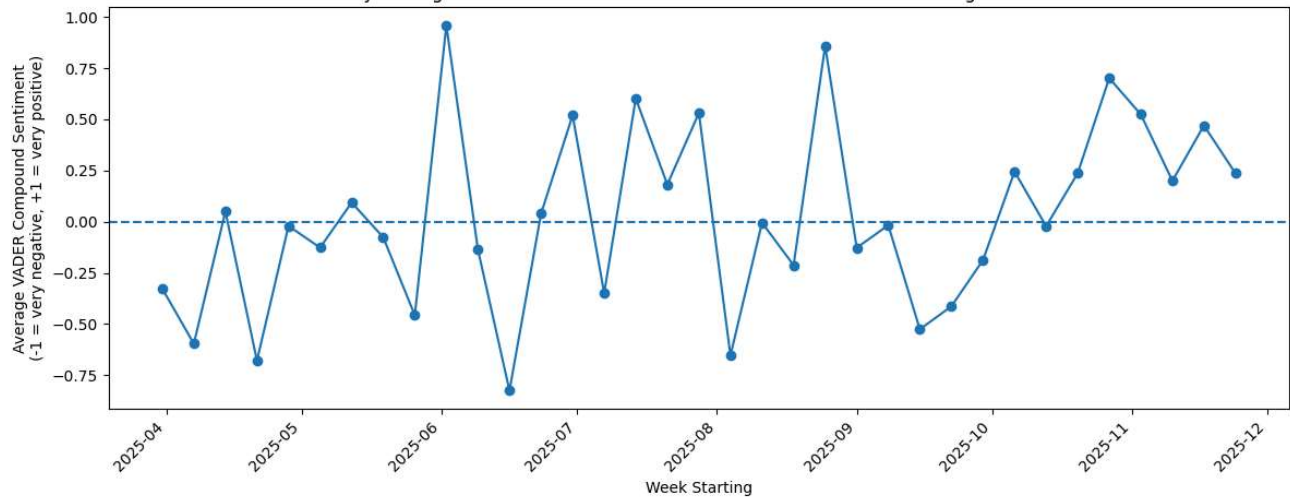
```

Weekly average VADER sentiment and article counts:

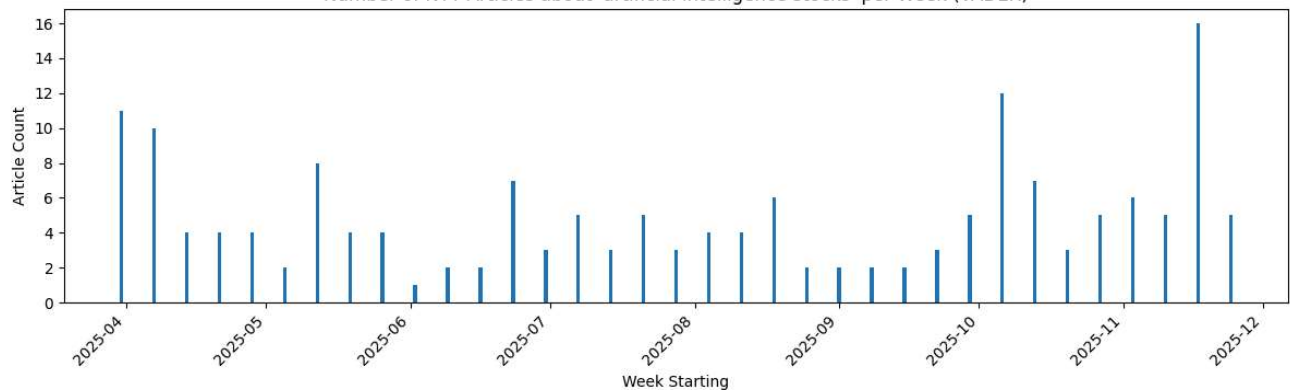
	week	avg_sentiment	article_count	week_start
0	2025-03-31/2025-04-06	-0.326973	11	2025-03-31
1	2025-04-07/2025-04-13	-0.596000	10	2025-04-07
2	2025-04-14/2025-04-20	0.049800	4	2025-04-14
3	2025-04-21/2025-04-27	-0.678400	4	2025-04-21
4	2025-04-28/2025-05-04	-0.020850	4	2025-04-28

/tmp/ipython-input-3784468423.py:11: UserWarning: Converting to PeriodArray/Index representation will drop timezone information
cleaned_df['week'] = cleaned_df['pub_date'].dt.to_period('W')

Weekly Average VADER Sentiment for NYT Articles about 'artificial intelligence stocks'



Number of NYT Articles about 'artificial intelligence stocks' per Week (VADER)



```
# =====
# 7-Day Moving Average (Rolling Mean) VADER Sentiment
# =====

import pandas as pd
import matplotlib.pyplot as plt

# Ensure date column exists
cleaned_df['pub_date'] = pd.to_datetime(cleaned_df['pub_date'])
cleaned_df['date'] = cleaned_df['pub_date'].dt.date

# Daily average first
daily_vader = (
    cleaned_df
    .groupby('date')['vader_compound']
    .mean()
    .reset_index()
    .rename(columns={'vader_compound': 'daily_sentiment'})
)
```

```
# Sort
daily_vader = daily_vader.sort_values('date')

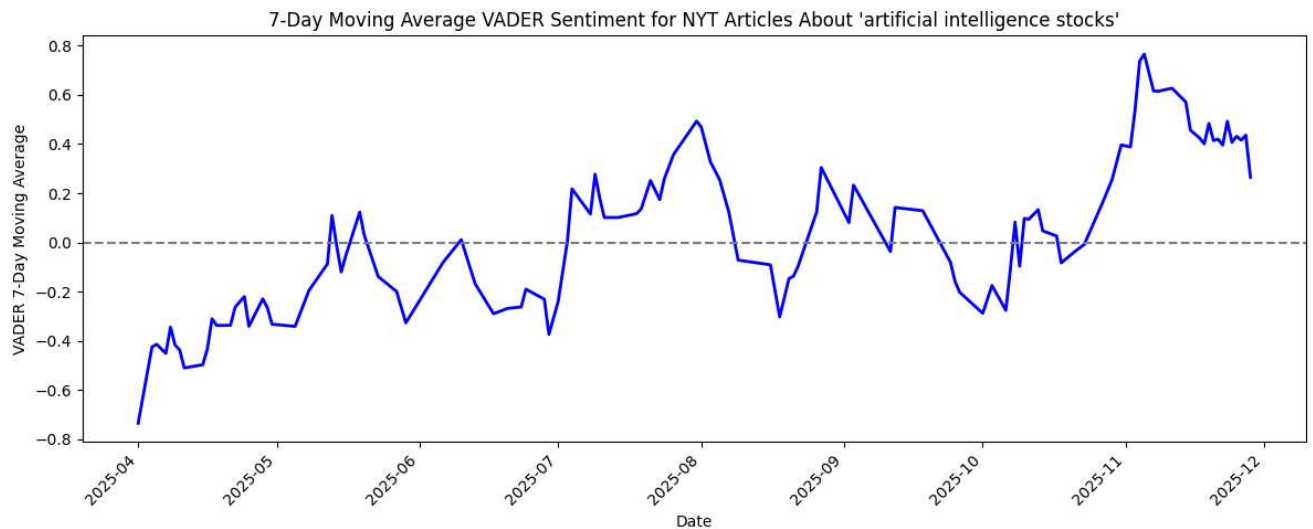
# 7-day moving average
daily_vader['vader_ma7'] = daily_vader['daily_sentiment'].rolling(window=7, min_periods=1).mean()

plt.figure(figsize=(12,5))
plt.plot(daily_vader['date'], daily_vader['vader_ma7'], color='blue', linewidth=2)

plt.axhline(0, linestyle='--', color='gray')

plt.title(f"7-Day Moving Average VADER Sentiment for NYT Articles About '{KEYWORD}'")
plt.xlabel("Date")
plt.ylabel("VADER 7-Day Moving Average")
plt.xticks(rotation=45, ha='right')

plt.tight_layout()
plt.show()
```



```
# =====
# 14-Day Moving Average VADER Sentiment
# =====

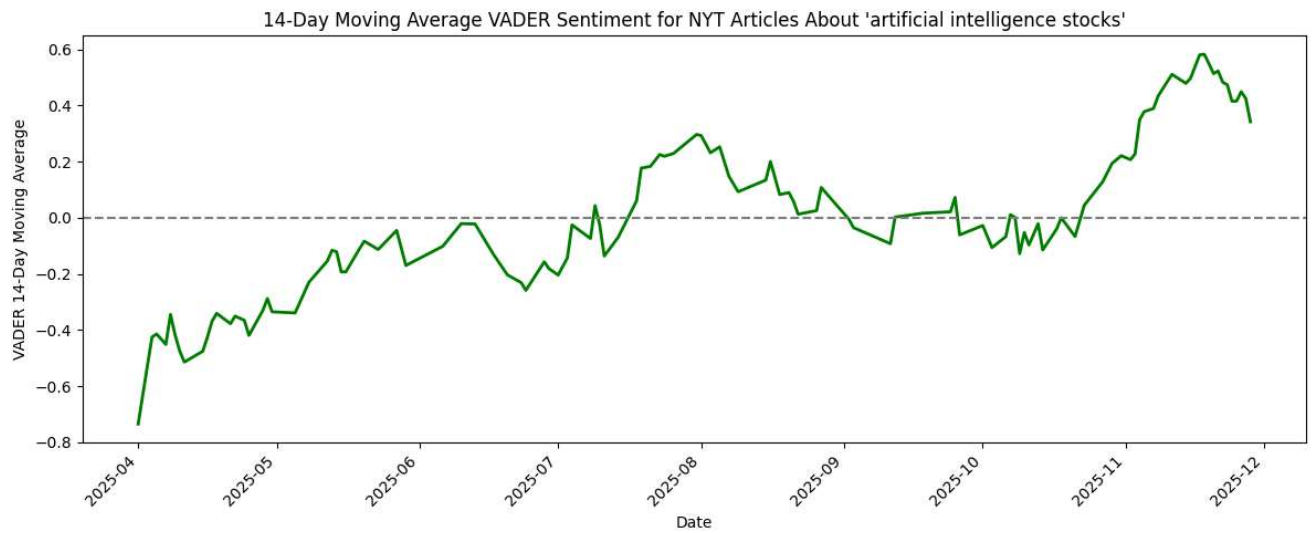
daily_vader['vader_ma14'] = daily_vader['daily_sentiment'].rolling(window=14, min_periods=1).mean()

plt.figure(figsize=(12,5))
plt.plot(daily_vader['date'], daily_vader['vader_ma14'], color='green', linewidth=2)

plt.axhline(0, linestyle='--', color='gray')

plt.title(f"14-Day Moving Average VADER Sentiment for NYT Articles About '{KEYWORD}'")
plt.xlabel("Date")
plt.ylabel("VADER 14-Day Moving Average")
plt.xticks(rotation=45, ha='right')

plt.tight_layout()
plt.show()
```



```
# =====
# 30-Day Moving Average VADER Sentiment
# =====

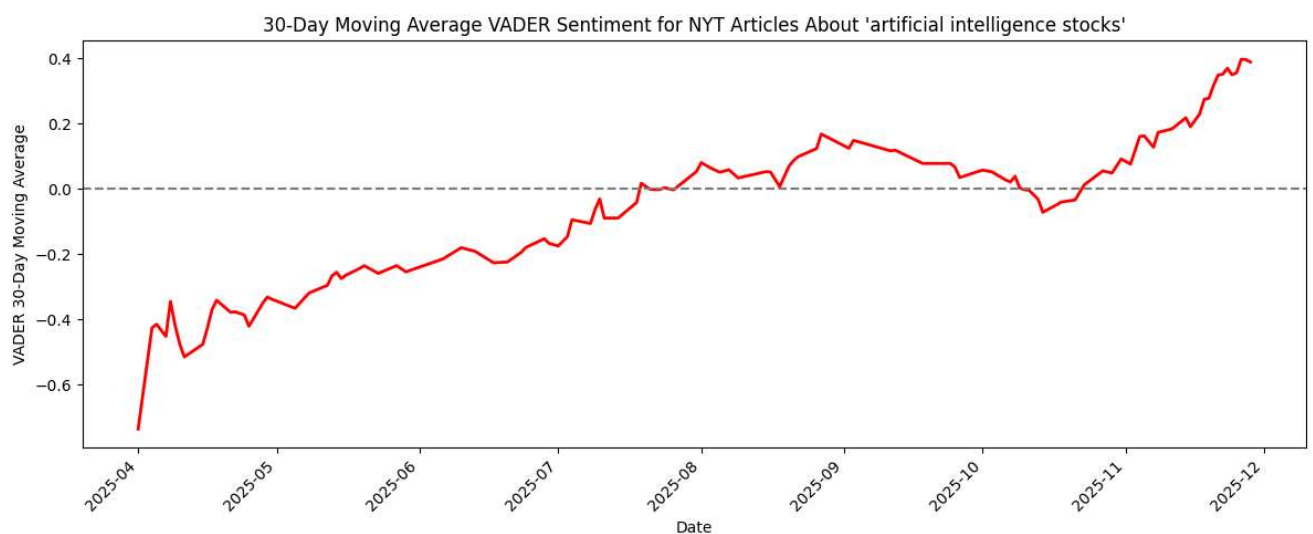
daily_vader['vader_ma30'] = daily_vader['daily_sentiment'].rolling(window=30, min_periods=1).mean()

plt.figure(figsize=(12,5))
plt.plot(daily_vader['date'], daily_vader['vader_ma30'], color='red', linewidth=2)

plt.axhline(0, linestyle='--', color='gray')

plt.title(f"30-Day Moving Average VADER Sentiment for NYT Articles About '{KEYWORD}'")
plt.xlabel("Date")
plt.ylabel("VADER 30-Day Moving Average")
plt.xticks(rotation=45, ha='right')

plt.tight_layout()
plt.show()
```



```
import seaborn as sns
```

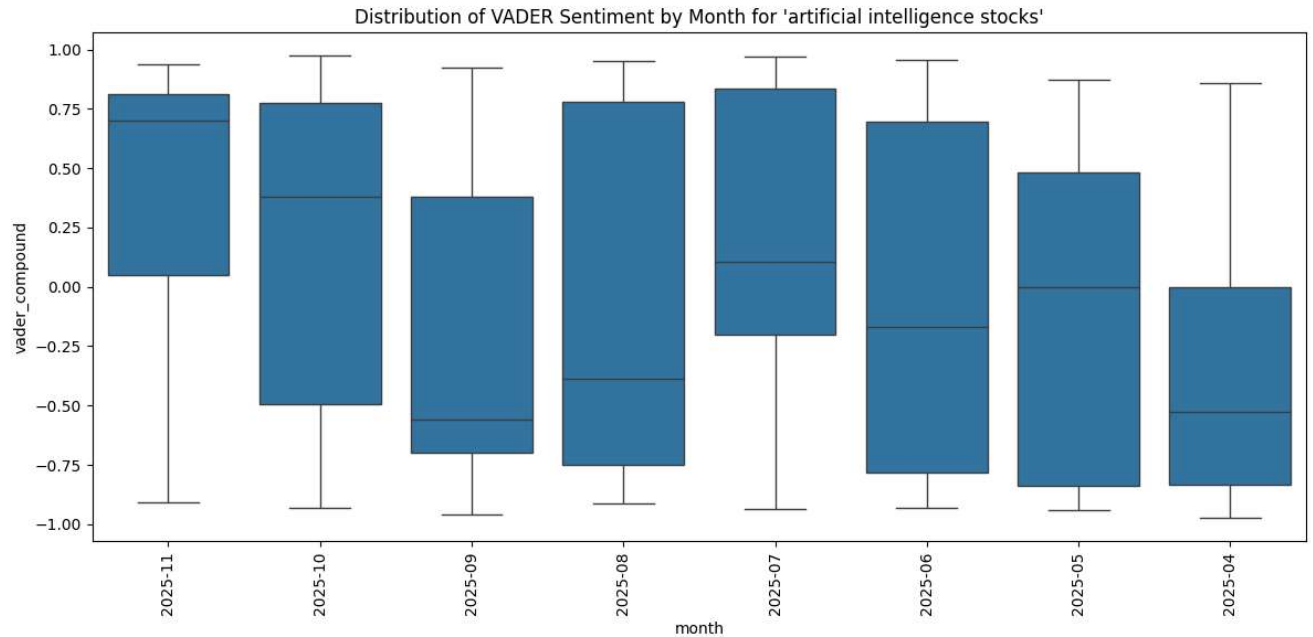
```

cleaned_df['month'] = cleaned_df['pub_date'].dt.to_period('M')

plt.figure(figsize=(12,6))
sns.boxplot(data=cleaned_df, x='month', y='vader_compound')
plt.xticks(rotation=90)
plt.title(f"Distribution of VADER Sentiment by Month for '{KEYWORD}'")
plt.tight_layout()
plt.show()

```

/tmp/ipython-input-1814252632.py:3: UserWarning: Converting to PeriodArray/Index representation will drop timezone information.
 cleaned_df['month'] = cleaned_df['pub_date'].dt.to_period('M')



```

# =====
# Volume-Weighted 7-Day Moving Average (VADER)
# =====

import pandas as pd
import matplotlib.pyplot as plt

# Ensure date column
cleaned_df['pub_date'] = pd.to_datetime(cleaned_df['pub_date'])
cleaned_df['date'] = cleaned_df['pub_date'].dt.date

# Compute both daily sentiment AND daily article count
daily_vader = (
    cleaned_df
    .groupby('date')['vader_compound']
    .agg(['mean', 'count'])
    .reset_index()
    .rename(columns={'mean': 'daily_sentiment', 'count': 'article_count'})
)

daily_vader = daily_vader.sort_values('date')

# ----- Volume-weighted 7-day moving average -----
window = 7

# numerator: sum(sentiment * volume)
daily_vader['weighted_sentiment'] = daily_vader['daily_sentiment'] * daily_vader['article_count']

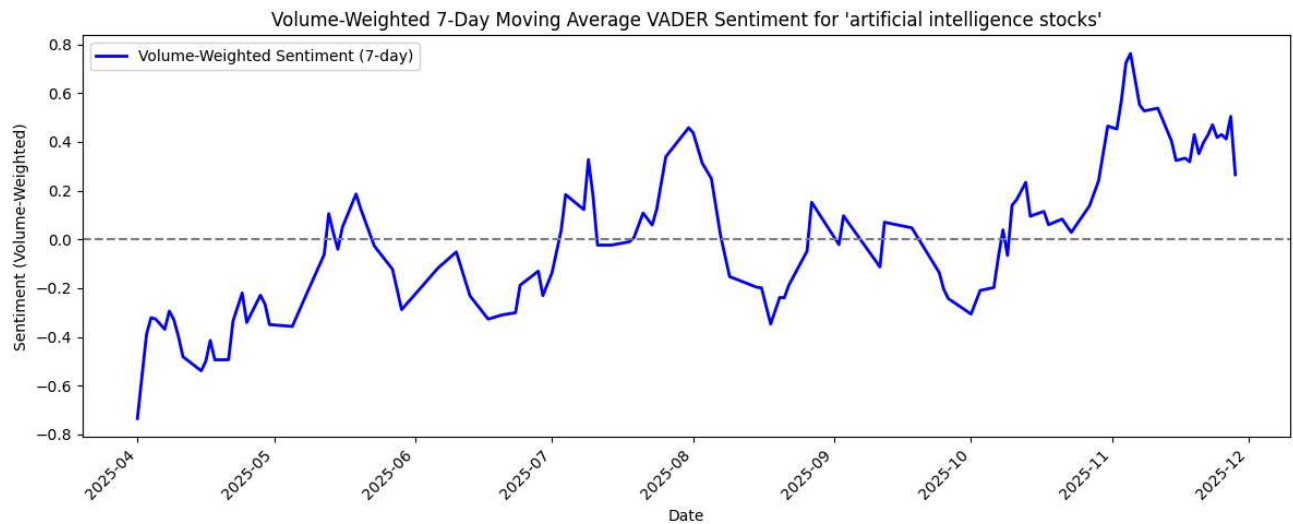
daily_vader['vw_ma7'] = (
    daily_vader['weighted_sentiment'].rolling(window=window, min_periods=1).sum()
    /
    daily_vader['article_count'].rolling(window=window, min_periods=1).sum()
)

```

```
)

# ----- Plot -----
plt.figure(figsize=(12,5))
plt.plot(
    daily_vader['date'],
    daily_vader['vw_ma7'],
    color='blue',
    linewidth=2,
    label="Volume-Weighted Sentiment (7-day)"
)

plt.axhline(0, linestyle='--', color='gray')
plt.title(f"Volume-Weighted 7-Day Moving Average VADER Sentiment for '{KEYWORD}'")
plt.xlabel("Date")
plt.ylabel("Sentiment (Volume-Weighted)")
plt.xticks(rotation=45, ha='right')
plt.legend()
plt.tight_layout()
plt.show()
print(daily_vader[['date', 'daily_sentiment', 'article_count']].head(20))
print(daily_vader['article_count'].value_counts())
```



	date	daily_sentiment	article_count
0	2025-04-01	-0.735100	1
1	2025-04-03	-0.320180	5
2	2025-04-04	-0.219725	4
3	2025-04-05	-0.381800	1
4	2025-04-07	-0.599150	2
5	2025-04-08	0.189650	2
6	2025-04-09	-0.855500	1
7	2025-04-10	-0.874433	3
8	2025-04-11	-0.831100	2
9	2025-04-15	-0.128000	1
10	2025-04-16	0.077200	1
11	2025-04-17	0.250000	1
12	2025-04-18	0.000000	1
13	2025-04-21	-0.851900	1
14	2025-04-22	-0.361200	1
15	2025-04-24	-0.526700	1
16	2025-04-25	-0.973800	1
17	2025-04-28	0.856900	1
18	2025-04-29	0.000000	1
19	2025-04-30	-0.470150	2

article_count

1 79

2 23

3 9

5 3

4 1

Name: count, dtype: int64

```
# =====
# Volume-Weighted 14-Day Moving Average (VADER)
# =====

import pandas as pd
import matplotlib.pyplot as plt

# Ensure date column
cleaned_df['pub_date'] = pd.to_datetime(cleaned_df['pub_date'])
cleaned_df['date'] = cleaned_df['pub_date'].dt.date

# Compute both daily sentiment AND daily article count
daily_vader = (
    cleaned_df
    .groupby('date')['vader_compound']
    .agg(['mean', 'count'])
    .reset_index()
    .rename(columns={'mean': 'daily_sentiment', 'count': 'article_count'})
)

daily_vader = daily_vader.sort_values('date')
```

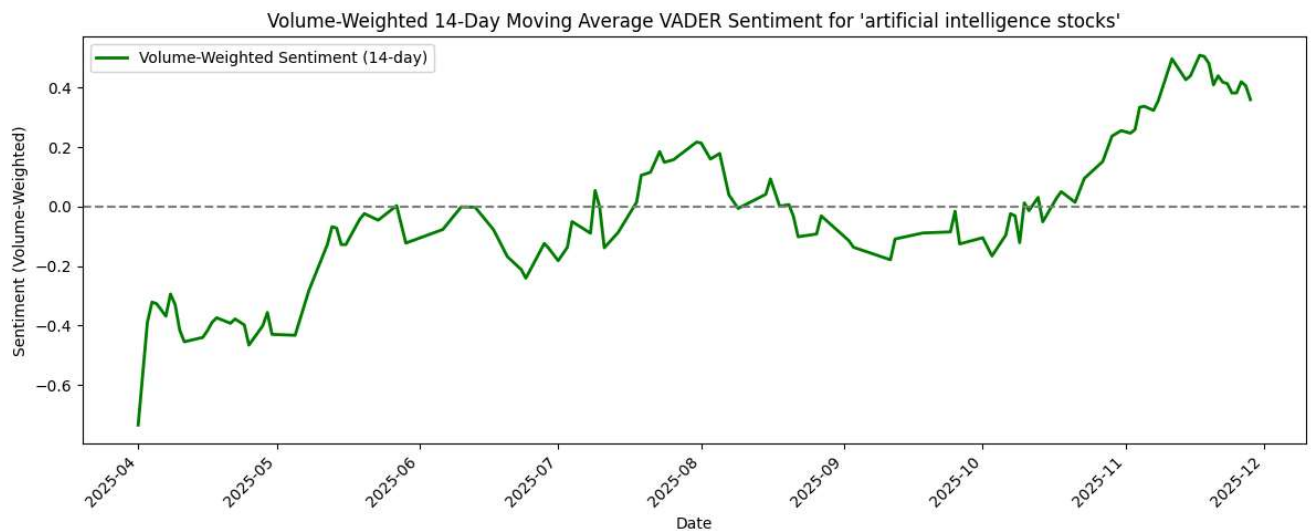
```
# ----- Volume-weighted 14-day moving average -----
window = 14

# numerator: sum(sentiment * volume)
daily_vader['weighted_sentiment'] = (
    daily_vader['daily_sentiment'] * daily_vader['article_count']
)

daily_vader['vw_ma14'] = (
    daily_vader['weighted_sentiment'].rolling(window=window, min_periods=1).sum()
    /
    daily_vader['article_count'].rolling(window=window, min_periods=1).sum()
)

# ----- Plot -----
plt.figure(figsize=(12,5))
plt.plot(
    daily_vader['date'],
    daily_vader['vw_ma14'],
    color='green',
    linewidth=2,
    label="Volume-Weighted Sentiment (14-day)"
)

plt.axhline(0, linestyle='--', color='gray')
plt.title(f"Volume-Weighted 14-Day Moving Average VADER Sentiment for '{KEYWORD}'")
plt.xlabel("Date")
plt.ylabel("Sentiment (Volume-Weighted)")
plt.xticks(rotation=45, ha='right')
plt.legend()
plt.tight_layout()
plt.show()
```



```
# =====
# Volume-Weighted 30-Day Moving Average (VADER)
# =====

import pandas as pd
import matplotlib.pyplot as plt

# Ensure date column
cleaned_df['pub_date'] = pd.to_datetime(cleaned_df['pub_date'])
cleaned_df['date'] = cleaned_df['pub_date'].dt.date

# Compute both daily sentiment AND daily article count
daily_vader = (
    cleaned_df
    .groupby('date')['vader_compound']
```

```

    .agg(['mean', 'count'])
    .reset_index()
    .rename(columns={'mean': 'daily_sentiment', 'count': 'article_count'})
)

daily_vader = daily_vader.sort_values('date')

# ----- Volume-weighted 30-day moving average -----
window = 30

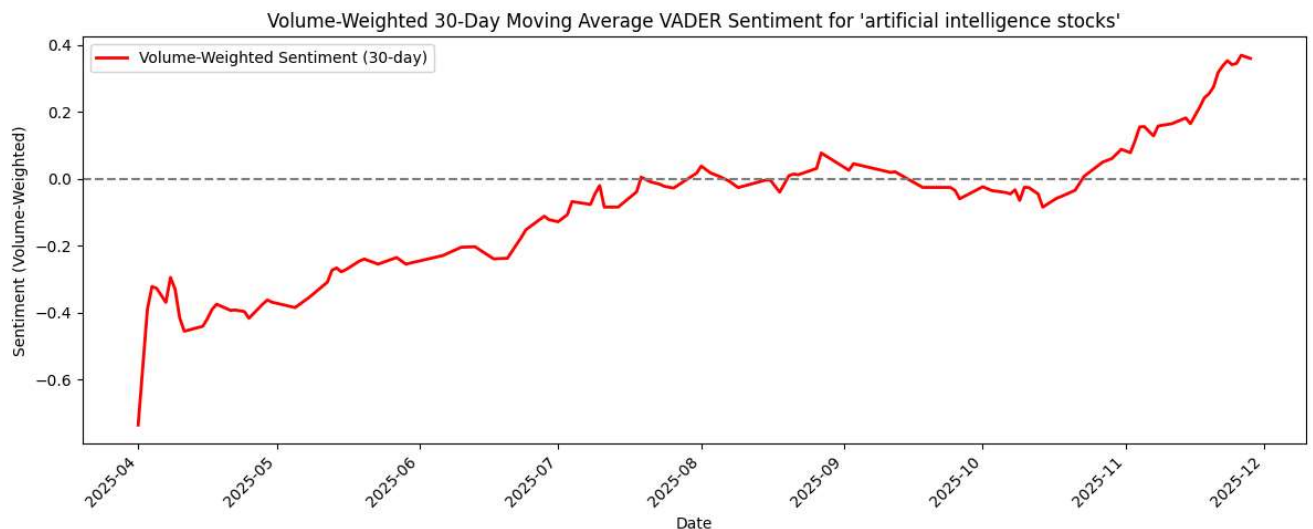
# numerator: sum(sentiment * volume)
daily_vader['weighted_sentiment'] = (
    daily_vader['daily_sentiment'] * daily_vader['article_count']
)

daily_vader['vw_ma30'] = (
    daily_vader['weighted_sentiment'].rolling(window=window, min_periods=1).sum()
    /
    daily_vader['article_count'].rolling(window=window, min_periods=1).sum()
)

# ----- Plot -----
plt.figure(figsize=(12,5))
plt.plot(
    daily_vader['date'],
    daily_vader['vw_ma30'],
    color='red',
    linewidth=2,
    label="Volume-Weighted Sentiment (30-day)"
)

plt.axhline(0, linestyle='--', color='gray')
plt.title(f"Volume-Weighted 30-Day Moving Average VADER Sentiment for '{KEYWORD}'")
plt.xlabel("Date")
plt.ylabel("Sentiment (Volume-Weighted)")
plt.xticks(rotation=45, ha='right')
plt.legend()
plt.tight_layout()
plt.show()

```



```

import pandas as pd
import matplotlib.pyplot as plt

# Ensure pub_date is datetime
cleaned_df["pub_date"] = pd.to_datetime(cleaned_df["pub_date"])

# ===== 1. Daily aggregates (avg sentiment + count) =====
daily_vader = (
    cleaned_df

```

```

.groupby(cleaned_df["pub_date"].dt.date)
.agg(
    avg_sentiment=("vader_compound", "mean"),
    article_count=("vader_compound", "size")
)
.reset_index()
.rename(columns={"pub_date": "date"})
)

daily_vader["date"] = pd.to_datetime(daily_vader["date"])
daily_vader = daily_vader.sort_values("date")

# ===== 2. 7-day moving averages =====
daily_vader["sentiment_ma7"] = (
    daily_vader["avg_sentiment"]
    .rolling(window=7, min_periods=1)
    .mean()
)
daily_vader["volume_ma7"] = (
    daily_vader["article_count"]
    .rolling(window=7, min_periods=1)
    .sum()
)

# ===== 3. 14-day moving averages =====
daily_vader["sentiment_ma14"] = (
    daily_vader["avg_sentiment"]
    .rolling(window=14, min_periods=1)
    .mean()
)
daily_vader["volume_ma14"] = (
    daily_vader["article_count"]
    .rolling(window=14, min_periods=1)
    .sum()
)

# ===== 4. 30-day moving averages =====
daily_vader["sentiment_ma30"] = (
    daily_vader["avg_sentiment"]
    .rolling(window=30, min_periods=1)
    .mean()
)
daily_vader["volume_ma30"] = (
    daily_vader["article_count"]
    .rolling(window=30, min_periods=1)
    .sum()
)

daily_vader.head()

```

	date	avg_sentiment	article_count	sentiment_ma7	volume_ma7	sentiment_ma14	volume_ma14	sentiment_ma30	volume_ma30
0	2025-04-01	-0.735100	1	-0.735100	1.0	-0.735100	1.0	-0.735100	1.0
1	2025-04-03	-0.320180	5	-0.527640	6.0	-0.527640	6.0	-0.527640	6.0
2	2025-04-04	-0.219725	4	-0.425002	10.0	-0.425002	10.0	-0.425002	10.0

Next steps: [Generate code with daily_vader](#) [New interactive sheet](#)

```

# ===== 7-day dual-axis plot: sentiment vs volume =====
fig, ax1 = plt.subplots(figsize=(12, 5))

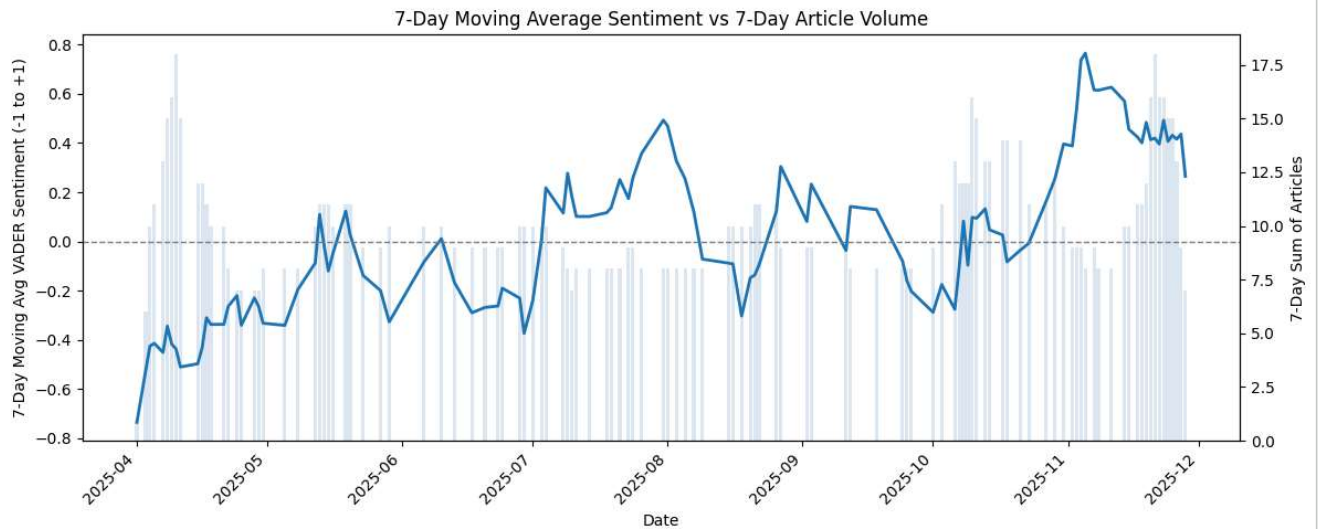
# Left y-axis: 7-day average sentiment (no markers, slightly thicker line)
ax1.plot(
    daily_vader["date"],
    daily_vader["sentiment_ma7"],
    linestyle="--",
    linewidth=2,
)
ax1.axhline(0, linestyle="--", linewidth=1, color="gray") # neutral baseline
ax1.set_xlabel("Date")

```

```
ax1.set_ylabel("7-Day Moving Avg VADER Sentiment (-1 to +1)")

# Right y-axis: 7-day summed article volume (lighter bars)
ax2 = ax1.twinx()
ax2.bar(
    daily_vader["date"],
    daily_vader["volume_ma7"],
    alpha=0.15
)
ax2.set_ylabel("7-Day Sum of Articles")

plt.title("7-Day Moving Average Sentiment vs 7-Day Article Volume")
fig.autofmt_xdate(rotation=45)
plt.tight_layout()
plt.show()
```

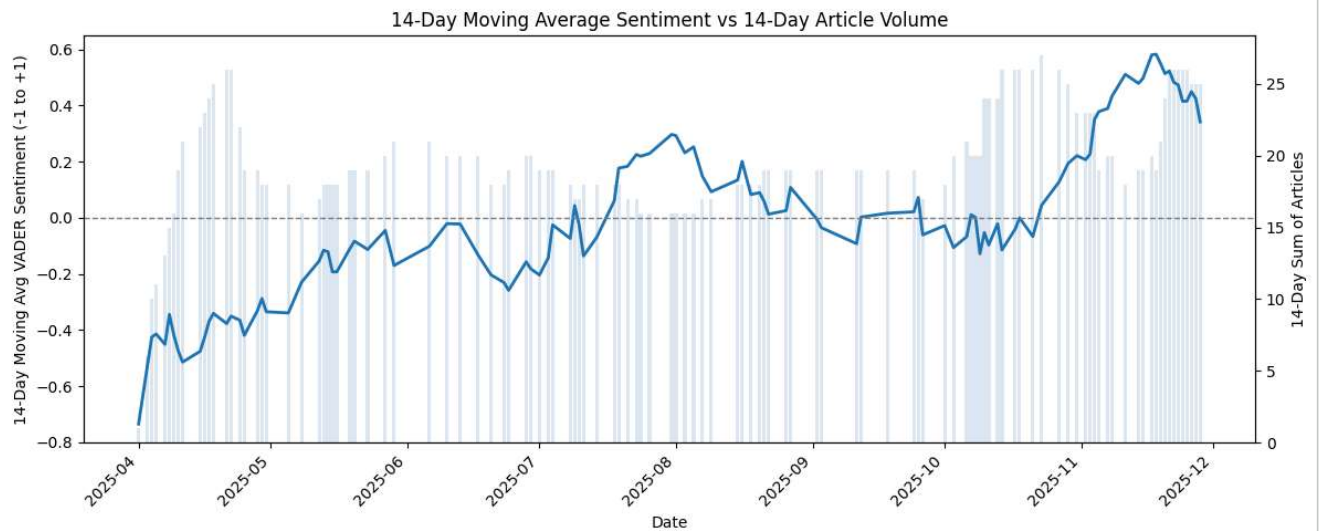


```
# ===== 14-day dual-axis plot: sentiment vs volume =====
fig, ax1 = plt.subplots(figsize=(12, 5))

# Left y-axis: 14-day average sentiment
ax1.plot(
    daily_vader["date"],
    daily_vader["sentiment_ma14"],
    linestyle="--",
    linewidth=2,
)
ax1.axhline(0, linestyle="--", linewidth=1, color="gray")
ax1.set_xlabel("Date")
ax1.set_ylabel("14-Day Moving Avg VADER Sentiment (-1 to +1)")

# Right y-axis: 14-day summed article volume
ax2 = ax1.twinx()
ax2.bar(
    daily_vader["date"],
    daily_vader["volume_ma14"],
    alpha=0.15
)
ax2.set_ylabel("14-Day Sum of Articles")

plt.title("14-Day Moving Average Sentiment vs 14-Day Article Volume")
fig.autofmt_xdate(rotation=45)
plt.tight_layout()
plt.show()
```

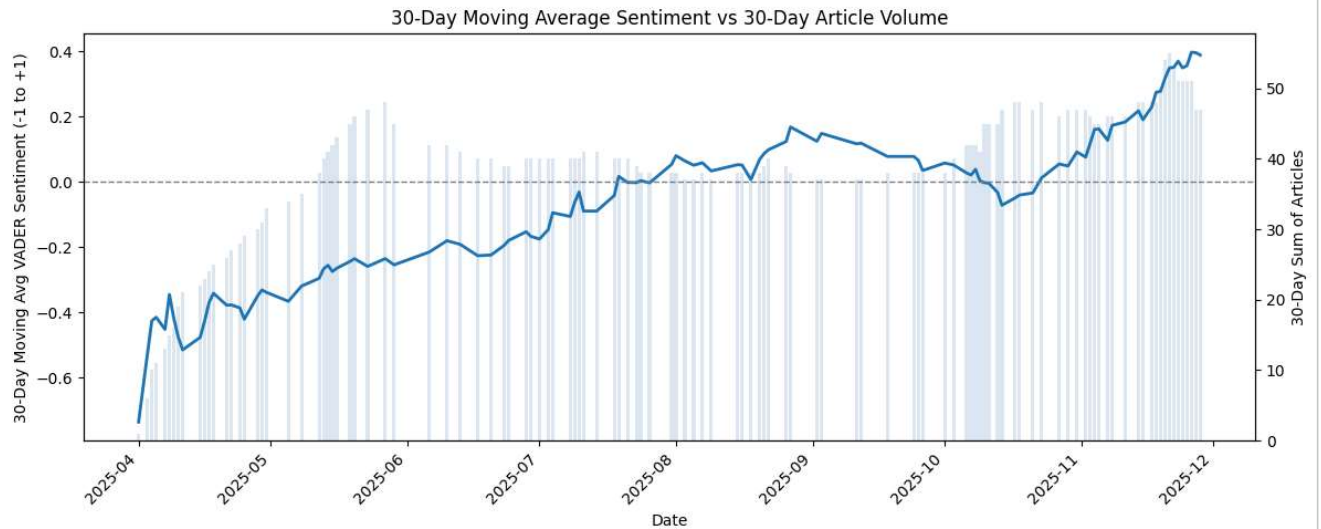


```
# ===== 30-day dual-axis plot: sentiment vs volume =====
fig, ax1 = plt.subplots(figsize=(12, 5))

# Left y-axis: 30-day average sentiment
ax1.plot(
    daily_vader["date"],
    daily_vader["sentiment_ma30"],
    linestyle="--",
    linewidth=2,
)
ax1.axhline(0, linestyle="--", linewidth=1, color="gray")
ax1.set_xlabel("Date")
ax1.set_ylabel("30-Day Moving Avg VADER Sentiment (-1 to +1)")

# Right y-axis: 30-day summed article volume
ax2 = ax1.twinx()
ax2.bar(
    daily_vader["date"],
    daily_vader["volume_ma30"],
    alpha=0.15
)
ax2.set_ylabel("30-Day Sum of Articles")

plt.title("30-Day Moving Average Sentiment vs 30-Day Article Volume")
fig.autofmt_xdate(rotation=45)
plt.tight_layout()
plt.show()
```



```
import pandas as pd
import matplotlib.pyplot as plt

# Ensure pub_date is datetime
cleaned_df["pub_date"] = pd.to_datetime(cleaned_df["pub_date"])

# ===== 1. Daily aggregates (avg sentiment + count) =====
daily_vader = (
    cleaned_df
    .groupby(cleaned_df["pub_date"].dt.date)
    .agg(
        avg_sentiment=("vader_compound", "mean"),
        article_count=("vader_compound", "size")
    )
    .reset_index()
    .rename(columns={"pub_date": "date"})
)

daily_vader["date"] = pd.to_datetime(daily_vader["date"])

# ===== 2. 7-day moving averages =====
daily_vader = daily_vader.sort_values("date")

daily_vader["sentiment_ma7"] = (
    daily_vader["avg_sentiment"]
    .rolling(window=7, min_periods=1)
    .mean()
)

# you can use sum or mean for volume; here we use 7-day sum
daily_vader["volume_ma7"] = (
    daily_vader["article_count"]
    .rolling(window=7, min_periods=1)
    .sum()
)

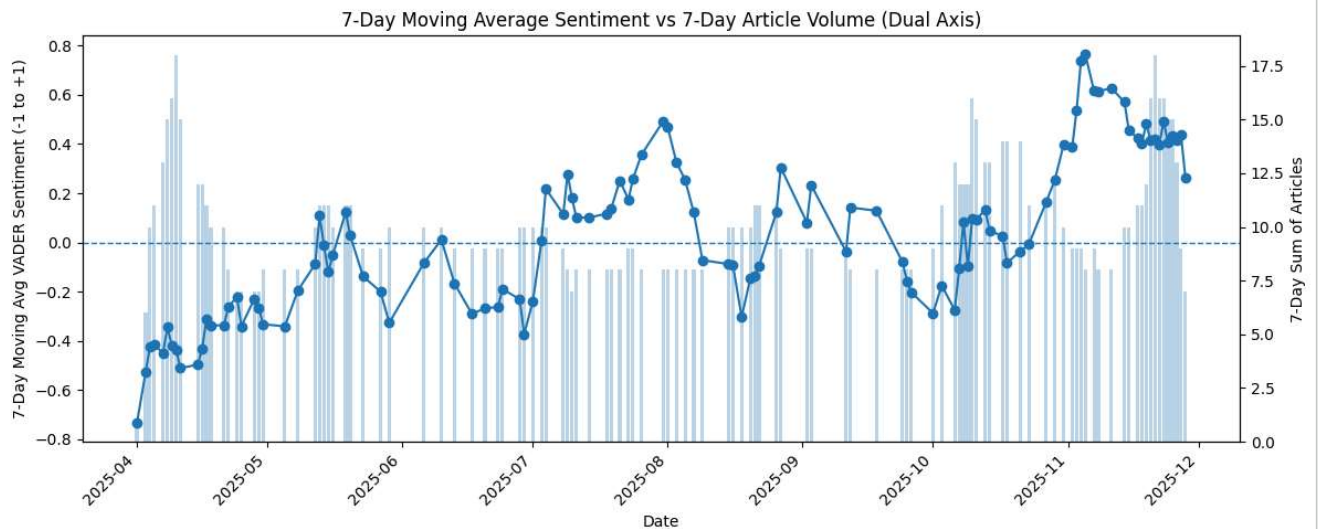
# ===== 3. Dual-axis plot: 7-day MA sentiment vs 7-day volume =====
fig, ax1 = plt.subplots(figsize=(12, 5))

# Left y-axis: 7-day average sentiment
ax1.plot(
    daily_vader["date"],
    daily_vader["sentiment_ma7"],
    linestyle="--",
    marker="o",
)

ax1.axhline(0, linestyle="--", linewidth=1) # neutral baseline
ax1.set_xlabel("Date")
ax1.set_ylabel("7-Day Moving Avg VADER Sentiment (-1 to +1)")
```

```
# Right y-axis: 7-day summed article volume
ax2 = ax1.twinx()
ax2.bar(
    daily_vader["date"],
    daily_vader["volume_ma7"],
    alpha=0.3
)
ax2.set_ylabel("7-Day Sum of Articles")

plt.title("7-Day Moving Average Sentiment vs 7-Day Article Volume (Dual Axis)")
fig.autofmt_xdate(rotation=45)
plt.tight_layout()
plt.show()
```



```
# ===== 14-day moving averages =====
daily_vader["sentiment_ma14"] = (
    daily_vader["avg_sentiment"]
    .rolling(window=14, min_periods=1)
    .mean()
)

daily_vader["volume_ma14"] = (
    daily_vader["article_count"]
    .rolling(window=14, min_periods=1)
    .sum()
)

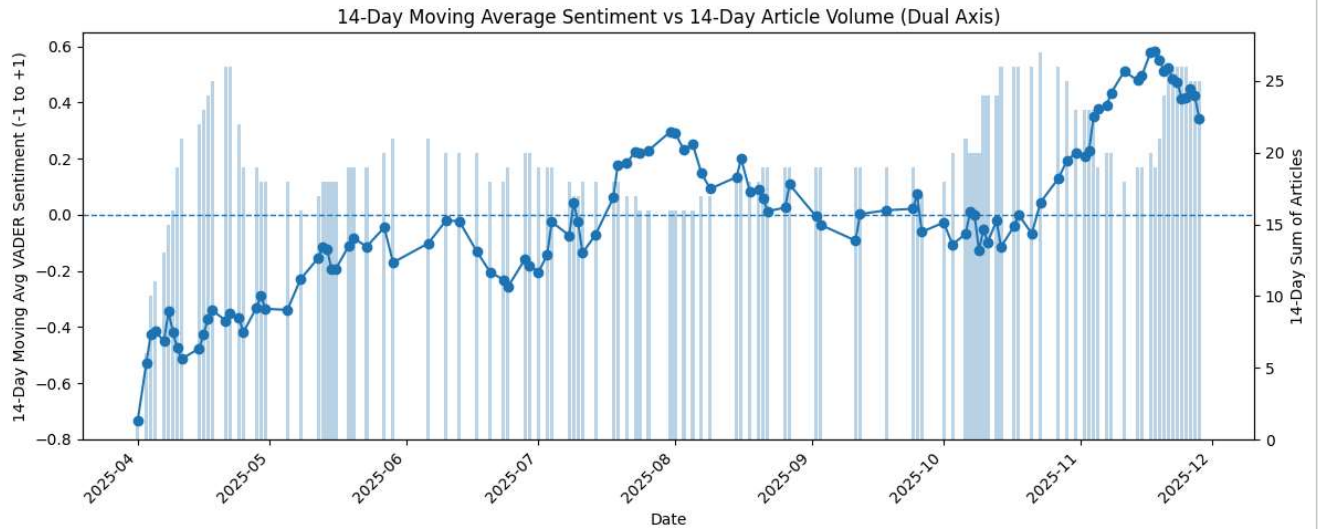
# ===== Dual-axis plot: 14-day MA sentiment vs 14-day volume =====
fig, ax1 = plt.subplots(figsize=(12, 5))

# Left y-axis: 14-day average sentiment
ax1.plot(
    daily_vader["date"],
    daily_vader["sentiment_ma14"],
    linestyle="--",
    marker="o",
)
ax1.axhline(0, linestyle="--", linewidth=1) # neutral baseline
ax1.set_xlabel("Date")
ax1.set_ylabel("14-Day Moving Avg VADER Sentiment (-1 to +1)")

# Right y-axis: 14-day summed article volume
ax2 = ax1.twinx()
ax2.bar(
    daily_vader["date"],
    daily_vader["volume_ma14"],
    alpha=0.3
)
```

```
ax2.set_ylabel("14-Day Sum of Articles")
```

```
plt.title("14-Day Moving Average Sentiment vs 14-Day Article Volume (Dual Axis)")
fig.autofmt_xdate(rotation=45)
plt.tight_layout()
plt.show()
```



```
# ===== 30-day moving averages =====
daily_vader["sentiment_ma30"] = (
    daily_vader["avg_sentiment"]
    .rolling(window=30, min_periods=1)
    .mean()
)

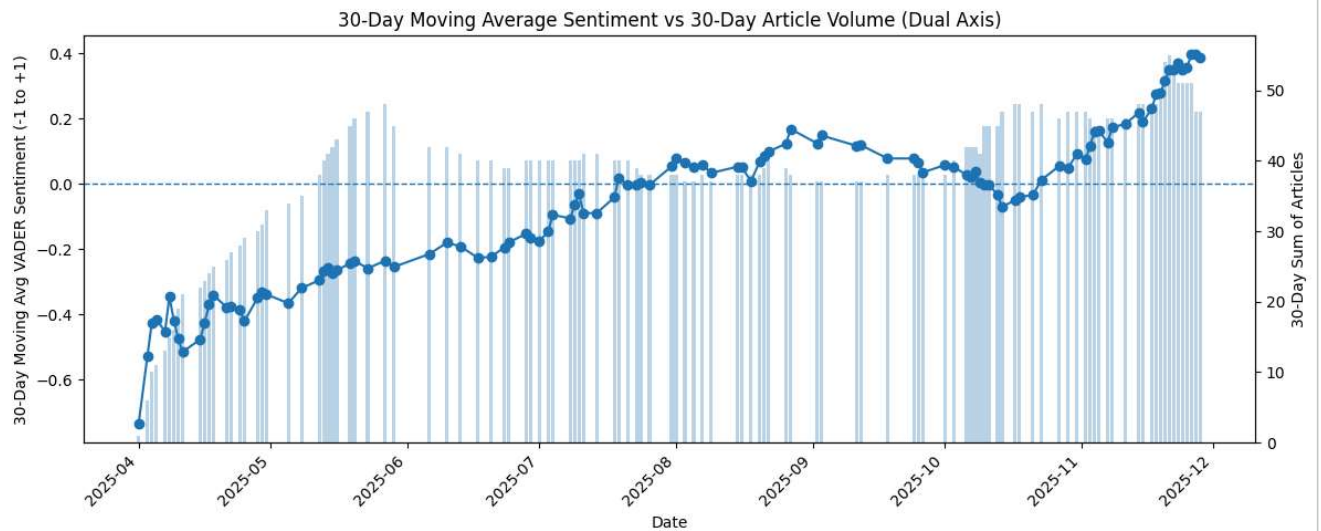
daily_vader["volume_ma30"] = (
    daily_vader["article_count"]
    .rolling(window=30, min_periods=1)
    .sum()
)

# ===== Dual-axis plot: 30-day MA sentiment vs 30-day volume =====
fig, ax1 = plt.subplots(figsize=(12, 5))

# Left y-axis: 30-day average sentiment
ax1.plot(
    daily_vader["date"],
    daily_vader["sentiment_ma30"],
    linestyle="--",
    marker="o",
)
ax1.axhline(0, linestyle="--", linewidth=1) # neutral baseline
ax1.set_xlabel("Date")
ax1.set_ylabel("30-Day Moving Avg VADER Sentiment (-1 to +1)")

# Right y-axis: 30-day summed article volume
ax2 = ax1.twinx()
ax2.bar(
    daily_vader["date"],
    daily_vader["volume_ma30"],
    alpha=0.3
)
ax2.set_ylabel("30-Day Sum of Articles")

plt.title("30-Day Moving Average Sentiment vs 30-Day Article Volume (Dual Axis)")
fig.autofmt_xdate(rotation=45)
plt.tight_layout()
plt.show()
```



```
import pandas as pd
import matplotlib.pyplot as plt

# Ensure pub_date is datetime
cleaned_df["pub_date"] = pd.to_datetime(cleaned_df["pub_date"])

# ===== 1. Daily aggregates: average sentiment + raw daily volume =====
daily_vader = (
    cleaned_df
    .groupby(cleaned_df["pub_date"].dt.date)
    .agg(
        avg_sentiment=("vader_compound", "mean"),
        article_count=("vader_compound", "size")
    )
    .reset_index()
    .rename(columns={"pub_date": "date"})
)

daily_vader["date"] = pd.to_datetime(daily_vader["date"])
daily_vader = daily_vader.sort_values("date")

# ===== 2. Compute 7-, 14-, and 30-day sentiment moving averages =====
for window in [7, 14, 30]:
    daily_vader[f"sentence_ma{window}"] = (
        daily_vader["avg_sentiment"]
        .rolling(window=window, min_periods=1)
        .mean()
    )

# ===== 3. Dual-axis plot: three MAs vs RAW daily volume =====
fig, ax1 = plt.subplots(figsize=(12, 5))

# Left y-axis: smoothed sentiment (three windows)
ax1.plot(
    daily_vader["date"],
    daily_vader["sentence_ma7"],
    linestyle="--",
    linewidth=2,
    label="7-Day MA Sentiment",
)
ax1.plot(
    daily_vader["date"],
    daily_vader["sentence_ma14"],
    linestyle="--",
    linewidth=2,
    label="14-Day MA Sentiment",
)
ax1.plot(
    daily_vader["date"],
```